Operating Systems — Assignment 7

xv6: Threads, Semaphores, Deadlock Detection and Priority Scheduling

Question 1 — Threads using clone() and join()

Task: Add thread support to xv6 so multiple threads can run in the same address space. Implement clone() to create a thread and join() to collect a finished thread.

What I did: I changed the process structure to identify threads and their leader. clone() shares the page directory with the current process but gives the new thread its own process entry, kernel stack and trap frame. I also updated exit(), wait() and join() so a shared address space is not freed while another thread is still using it.

Outcome: I tested three threads, each incrementing a shared counter 100,000 times without a lock. The final count was lower than 300,000 in every run (168,205; 167,765; 172,767; 169,451; and 166,007). This showed the race condition caused by simultaneous updates. After widening the gap between the load and store operations, the race became easier to reproduce.

Issue observed: If the leader exits while threads are still running, the shared address space must not be freed too early. I handled this by marking the remaining threads as killed and delaying address-space cleanup until no live thread remains. A limitation is that heap growth after clone() is not reflected in threads that already exist.

Question 2 — Semaphores

Task: Implement semaphores in xv6 and use them to protect shared data. Also test the producer-consumer problem with a bounded buffer.

What I did: I added a table of ten semaphores. Each semaphore has a value, a lock and a wakeup-ticket count. sem_wait() decreases the value and blocks when necessary; sem_signal() increases it and wakes a waiter. The ticket count is needed because xv6's wakeup() can wake multiple sleepers, while one signal should allow only one waiting operation to proceed.

Outcome — shared counter: Without synchronization, threadtest produced values around 166,000–173,000 instead of 300,000. With sem_wait() and sem_signal() protecting each increment in synctest, the counter reached exactly 300,000.

Outcome — producer-consumer: I tested a buffer of size five with 15 items. The producer waited when the buffer became full, and the consumer waited when it was empty. All 15 items were consumed in FIFO order, and the buffer did not overflow or underflow.

Question 3 — Deadlock Detection and Avoidance

Task: Implement deadlock detection and a Banker-style safety check. Test them with resource-allocation examples and with a real deadlock involving pipes.

What I did: The program keeps Allocation, Request, Max and Available data. The detection routine checks which processes can finish with the resources currently available and returns their resources to the Work vector. Any processes that remain unfinished are reported as deadlocked. For avoidance, the program checks a request against the process's Need and Available resources, temporarily grants it, and then checks whether the resulting state is safe. If it is unsafe, the request is rolled back.

Outcome — test cases: In the first example, P1, P2, P3 and P4 were reported as deadlocked while P0 could finish. In the second example, changing P2's request removed the deadlock. The initial Banker example was safe with Available = <3, 3, 2>. P1's request <1, 0, 2> was granted, P4's request <3, 3, 0> had to wait, and P0's request <0, 2, 0> was denied because it made the state unsafe.

Outcome — pipe deadlock: Process A held one pipe's write end and waited to read from another pipe, while process B held the other write end and waited in the opposite direction. Neither could continue. The monitor detected P0 and P1 as deadlocked. Pressing Ctrl-P showed both processes sleeping inside piperead, confirming that they were blocked. The monitor then killed both processes.

Issue observed: A killed child exits after read() returns -1, so it cannot print a final message itself. The monitor reports the deadlock instead.

Question 4 — Priority Scheduling with Aging

Task: Modify the scheduler so it chooses processes according to priority and uses aging to prevent low-priority processes from waiting indefinitely.

What I did: I added priority and waitticks fields to the process structure. Priority values range from 1 (highest) to 20 (lowest), and new processes start at 10. The scheduler selects the runnable process with the smallest priority number. Equal-priority processes get turns through a circular scan. On each timer tick, a waiting runnable process gains one wait tick; after more than 30 ticks, its priority improves by one level, and its waiting counter resets.

Outcome: I tested four CPU-bound child processes on one CPU with starting priorities 2, 6, 12 and 20. They finished in that order at ticks 62, 129, 191 and 254. The lowest-priority process eventually improved from priority 20 to 14 while waiting, so it was able to finish instead of waiting forever.

Issue observed: Initially, each child reset its own priority when it first ran, undoing the improvement earned through aging. I changed the test so the parent sets each child's priority immediately after fork(). I also used one CPU because running on two CPUs made the completion order harder to observe.

Conclusion

The tests showed the main purpose of each part: threads shared memory, but unsynchronized updates lost increments; semaphores protected the shared counter and coordinated the producer and consumer; the deadlock routines identified blocked processes and rejected an unsafe resource request; and priority aging allowed a low-priority process to make progress. The screenshots for these tests are kept in the ss/ folder.